

Learning the books — 82 lessons that were already written, and nobody could read

For Michael. Written at the end of books-44, slice C.

You asked for this:

"i want to be empowered to use these lessons and guidance to help make me the best possible accountant i can be. i don't need to know how to account for any other business, so all the complexity of trying to learn everything from every industry from every size like in school. i need to know everything there is to know about accounting for greenway."

That sentence changed what got built, so I want to start by telling you what it ruled out. A textbook teaches you accounting and then leaves you to work out which parts apply to you. That is the right design when the reader might run a hospital, a shipyard or a coffee shop. It is the wrong design when there is exactly one business, it is yours, and you already have the degree — you have a Master's in accounting from the University of Washington and you have not used it in thirteen years. You do not need the map of every country. You need the streets around your own shop, in the order you drive them.

So the course is organised by **when the work happens at Greenway**, not by subject. Not "assets, liabilities, equity, revenue, expenses." Instead: how money is written down, how the agencies count days, hiring someone, running a payroll, checking your own work, closing the month, the S-corporation year, and what it costs when something is late. That is the year, in order.

The uncomfortable part first

Before this slice, there were **82 finished lessons** in this system.

They were written. They were reviewed. They cited the actual statutes. They had tests, and the tests passed. And **not one of them was reachable from any screen in the application**. Not one page imported them. There was no link, no menu entry, no URL. You could not have read them if you had gone looking.

Six files, 2,100 lines, every test green, teaching nobody.

I want to be plain about what that means rather than dress it up. The tests were not lying — the lessons really were well-formed and internally consistent. The tests were answering a question nobody had connected to you. "Is this lesson correctly written?" is a different question from "can Michael read this lesson?", and only the first one was ever asked.

In this repository that failure has a name and a number — standing rule 50, *dead code wearing a green check* — because it has now happened enough times to be a category rather than an accident. It happened with the garnishment engine. It happened with net pay and year-to-date. It happened with the four financial statements: 1,600 lines, 132 passing tests, imported by nothing. Each time, the work was correct and invisible.

This one was the largest, and it was the worst kind, because what was buried was the teaching itself. You asked to be made a better accountant, and the material to do it had been sitting in the repository the whole time.

What now exists

A screen at **Accounting → Learning the Books**. Eight units, 82 lessons, in the order Greenway's year actually raises them.

#	Unit	Lessons	The question it answers
1	How money is written down	7	Why does the system refuse numbers that look perfectly reasonable?
2	How the agencies count days	14	Everyone says "the 20th" or "three business days". What do they actually mean?
3	Hiring someone — the paperwork before the first cheque	22	Somebody starts on Monday. What has to be true before I can pay them?
4	Running a payroll — every other Friday	4	How does a timesheet become the number on the cheque?
5	Checking your own work — before anyone else does	5	How do I know the quarter is right before I file it?
6	Closing the month — making a period stop meaning	8	What does it actually mean to say a month is finished?

6	Closing the month — making a period stop moving	8	What does it actually mean to say a month is finished?
7	The S-corporation year — where the shop's numbers become yours	5	Why does an S corporation need to know which year it started being one?
8	When something is late — penalties and interest	17	It slipped. What does it cost, and what do I do now?
Total		82	

Every unit tells you **why it sits where it sits**. Unit 1 is first because every other unit's arithmetic depends on knowing that money is an integer number of cents. Unit 8 is last because penalties are what happens when the first seven did not get done — reading about penalties before you have read about deadlines teaches fear instead of competence.

Each lesson opens into four blocks, and **the colour of a block always means the same thing**, on every card, in every unit:

- **Plain white — what it does.** The mechanics.
- **Gold — why this exists at all.** The reason the rule is there. Usually a statute, occasionally somebody's expensive mistake.
- **Orange — what goes wrong here.** The trap. Specific, not general.
- **Green — what I would do.** The recommendation, stated as a recommendation.

You told me once that the verbatim quote panels were "*hard to digest as there is a wall of words and color*", and that you learn best visually. That is why lessons are **collapsed by default** — you see a one-line summary and open only what you want. It is why each unit has its own colour so you always know where you are. And it is why colour carries **meaning** rather than decoration: once you learn that orange is the trap, you can skim eight lessons for orange and read nothing else.

Where a lesson rests on somebody else's words — a statute, a regulation, an IRS publication — those words appear in an amber panel, quoted exactly. **Amber is always someone else's language and never my commentary.** 61 different authorities are cited across the course, and a test checks every single citation resolves to a real document. Sixteen of the 82 lessons cite nothing, because they are teaching arithmetic or judgement rather than law; those say so explicitly rather than leaving a blank space where you would wonder if something failed to load.

The number at the top of the screen is not typed

The screen says "82 lessons in 8 units". **Nobody wrote 82 anywhere.** It is counted from the lesson files every time the page loads.

This matters more than it sounds. If the count were typed, it would be correct on the day it was typed and wrong forever afterwards, and it would never once go red to tell you. Someone adds a lesson: the screen still says 82, and the new lesson is invisible. That is precisely the failure this whole slice exists to end, quietly reintroduced by a hard-coded number.

So there is a test whose only job is to search all three source files for the literal `82` in executable code and fail if it finds one. It allows the number in comments, where it is describing history. If a lesson is ever added to any of the six teaching files, it appears in the course automatically and the count moves by itself.

There is also a **coverage banner** at the top of the screen. It is green when every lesson that exists is placed in the course. If one is ever written and not placed, the banner turns orange and **names it**. I could have made that impossible instead — refused to build at all — but then the banner would be a component that can never display anything, which is the same disease in a new coat.

I attacked it, and it did not survive intact

The tests passing is not evidence that they work. A test that cannot fail is not a test, and this repository has been bitten by that specific illusion often enough that I now try to break every gate I write.

So I wrote `scripts/prove-learning-path-gate.sh`, which makes **27 deliberate sabotages** of the new code and requires the test suite to catch each one: delete a lesson from the course, delete a whole unit, teach the same lesson twice, point the curriculum at a function that does not exist, swap the colour of "the trap" with the colour of "what I would do", truncate the teaching to sixty characters, cite an authority that does not exist, remove the screen from the menu, break the next/previous chain so the course dead-ends halfway.

The first run scored 19 out of 21, and the two survivors were both real holes.

Hole one: the security gate. Every accounting screen in this system is behind `requireBooksAccess()` — your recorded position is "there is no reason anyone else needs to see my books or my financials ever." My test checked for it like this:

the page file contains the text `requireBooksAccess`

I deleted the actual call from the page. The test stayed green — because the **import line at the top of the file** still contained that text. I had written a test that was satisfied by an unused import. An accounting screen with its security removed, and a suite that said everything was fine.

Hole two, and this one is worse. There is a function whose entire job is to verify the curriculum is not broken — no missing lessons, no duplicates, no empty units. I replaced its whole body with `return;` so it did nothing at all, and **the suite stayed green.**

The reason is worth sitting with, because it is a trap that catches good accountants too. The only thing my tests could do with that function was call it on the real curriculum and assert it did not complain. But **a function that does nothing never complains.** "It raised no objection" and "it found no problem" look identical from the outside, and I had been treating the first as evidence of the second for the entire slice.

It is the same reason you do not test a smoke alarm by observing that it is silent. You have to hold something under it.

Both are fixed. The security test now requires the awaited *call*, and I added a second attack — keep the call but remove the `await`, which would let the page render before the check finishes — to make sure the fix is real. The two verification functions were rewritten so the tests can hand them a deliberately broken curriculum and require them to object to each specific fault, one at a time. Then I re-ran the campaign: **27 attacks, 27 caught.**

The three attacks that were supposed to fail

A kill rate of 27 out of 27 is meaningless on its own, and I would rather you distrust that number than accept it.

If I had written a test so brittle that it went red whenever anyone touched anything, it would have scored a perfect 27 and told you nothing whatsoever. It would have been measuring its own paranoia and reporting it as rigour.

So the campaign also runs three **controls** — edits that genuinely change nothing that matters, where the test is required to stay **green**: adding an ordinary comment, rewording a unit's explanation, and swapping the order of two lessons within a single unit. That last one is deliberate. The course guarantees that every lesson appears exactly once and that the *units* run in a defended order; it deliberately does **not** freeze the sequence inside a unit, because otherwise nobody could ever improve the teaching without a test stopping them.

All three stayed green. **That is the number I would ask about first if someone showed me this work.** When a system catches everything you throw at it, the right next question is never "how many did it catch" — it is "*and what did it correctly allow?*" A control that rejects everything is not a control, it is an obstacle. This is the same instinct that makes a variance you cannot explain more worrying than one you can.

Two other things this slice found, which I am reporting rather than burying

An unused import that was actually a missing safety check. Four of the six teaching files had to be split in two this slice, for a technical reason: they read files from disk, and code that reads from disk cannot be sent to a web browser. So the lessons stayed in one file and the disk-reading checks moved to a second. Afterwards, the linter flagged a leftover unused import in two of the new files.

The five-second fix is to delete the unused import and watch the warning vanish. I nearly did. But that import was the **footprint of a check that had been lost in the split** — a check that every lesson is substantially written rather than a placeholder. Its two sibling files still had theirs; these two had silently dropped it. Deleting the import would have erased the only remaining evidence that something was missing. Both checks are restored, and both now have tests that deliberately break them to prove they work.

A roadmap figure that went stale the moment I did the work. The roadmap recorded these six files as 2,345 lines. Splitting them moved 245 lines out, so the true figure is now 2,100 — and a test that re-counts every roadmap number from the actual files caught the discrepancy on the next run. The roadmap is updated and explains why the number moved.

The lesson count did not change: 82 before the split, 82 after. That pairing is the point of tracking both figures. If teaching had been lost in a refactor, that is where it would have shown up.

What I would do with this, if I were you

Do not read it front to back. It is 82 lessons and it is not a book.

Read **Unit 1** properly, once. Seven lessons, and it explains why the system refuses numbers that look perfectly fine to you — that is the thing most likely to make you think something is broken when it is protecting you.

Then read **the one unit that matches what you are doing that week**. Hiring someone in March? Unit 3. First payroll on 1 January 2027? Unit 4, the week before. Closing January? Unit 6. That is what the "when you need this" line on each unit is for.

Use the search box when something confuses you in the moment. It searches the teaching, not just the function names, so typing "I-9" or "rounding" or "late" finds the lessons that discuss it even when the words are not in the title. If it finds nothing, it says so plainly and asks you to tell me — because a gap you can name is a gap I can close, and I would much rather hear "I searched for X and found nothing" than have you conclude the system does not cover it.

Every unit has a "**the one thing to remember**" line. If you read nothing else, read those eight sentences. They are the eight things that would cost you real money at Greenway.

What this does not do

It teaches you the accounting **for this business**, which is what you asked for and is a genuine limit rather than a modest one. It will not prepare you to account for a manufacturer or a non-profit. It does not cover consolidations, foreign currency, or leases, because Greenway does not have them.

It also does not file anything. That boundary has not moved: **we replace the data-preparation half of what Aatrix did. We do not become your filing agent.** The course teaches you what the numbers mean and where they come from; you or your CPA still submit them.

And four teaching modules are **still buried** — the ones covering cost of goods sold and §280E, the basis and AAA rules, the financial statements' own commentary, and internal control. I know exactly which four, they are named in the tests, and COGS/§280E is the next one to surface. I am telling you they are dark rather than quietly leaving them dark, which is the only difference between a known gap and the situation this slice just fixed.

Where things stand

We agreed the order **C → A → D → B**, and this was C.

A is next: finishing the W-2 and W-3 engine — boxes 1 through 6, the box 12 codes, boxes 15 through 20 with box 17 correctly left blank because Washington has no personal income tax, and the reconciliation between the W-3 and the four quarterly 941s.

B is still last and still blocked, waiting on the documents I have asked you for — Form 7203 for each shareholder, the Form 2553 and CP261, the ending AAA from Schedule M-2. Of everything on that list, the **2027 rates are the time-critical item**, because the first payroll runs on 1 January 2027 and the rates have to be in the system before it does.

Verified at the time of writing: 429 test files, 10,320 tests, all passing. 82 lessons, 8 units, 0 unplaced, 0 broken references. 27 sabotage attempts, 27 caught, 3 controls correctly allowed through.